

Reducing Hallucinations in Legal AI: Retrieval-Augmented Generation with a Gated Verification Chain

[Author 1]
Dept. of Artificial Intelligence,
[University]
[City, Country]
[email]

[Author 2]
Dept. of Artificial Intelligence,
[University]
[City, Country]
[email]

[Author 3]
Dept. of Artificial Intelligence,
[University]
[City, Country]
[email]

[Guide]
[Designation]
Dept. of Artificial Intelligence,
[University]
[City, Country]
[email]

Abstract

Large language models (LLMs) are increasingly used to answer legal questions, and they sometimes state rules or cite sources that do not exist. **Retrieval-augmented generation (RAG)** reduces this by grounding answers in retrieved documents, but it does not check the answer the model finally writes. This paper describes a legal question-answering system that verifies every answer before showing it. Documents are split with **summary-augmented chunking (SAC)** when they have statutory structure and by paragraph otherwise, stored in an embedded hybrid vector and keyword index, and retrieved with rank fusion and cross-encoder reranking. The model must cite a source passage for every claim, and a **six-layer verification chain** reduces its checks to a single **Verification Confidence Score (VCS)** that decides whether the answer is shown or withheld. On a labelled set of ten questions, SAC raised retrieval F1 from **0.333 to 0.417** over paragraph chunking. Of **28 deliberately corrupted claims**, the full chain showed **none** to the user; removing only the score gate showed **all 28**, while removing any single detection layer showed none. The same tests produced three false abstentions in ten answerable questions, which makes calibrating the gate the main open problem.

Keywords—legal AI, retrieval-augmented generation, hallucination, citation verification, natural language inference, abstention

I. INTRODUCTION

Legal work depends on sources. A statement about the law is only as good as the provision or judgment behind it, and whoever relies on it has to be able to check that source. General-purpose LLMs do not meet this standard reliably. Their answers read well, but some rest on citations that do not exist or on provisions that say something different. The consequences are real: in *Mata v. Avianca, Inc.* [1], a US

federal court sanctioned lawyers who filed a brief citing cases that a chatbot had invented, and an evaluation of commercial legal research tools found that products built on retrieval still hallucinate [2].

Retrieval-augmented generation [3] is the usual remedy: the model answers from passages retrieved from a trusted corpus instead of from memory alone. Three weaknesses survive it. Fixed-size chunking separates a clause from the section that gives it meaning. Retrieval ranks passages by similarity to the question, which is not the same as supporting the answer the model then writes. And nothing checks the generated answer against the evidence before a user reads it. Earlier work tackles these one at a time. Reuter et al. [4] introduced summary-augmented chunking for legal corpora, Figueiredo et al. [5] audit citations after generation, embedding-free approaches [6] move retrieval into the LLM at a higher cost per query, and benchmarks such as RAGTruth [7] and LegalBench-RAG [8] measure the problem without acting on it.

In this work we build a legal RAG system in which an answer has to pass verification before it is shown. Our contributions are:

- 1) An end-to-end pipeline that combines adaptive chunking, hybrid retrieval over an embedded LanceDB index, and citation-forced generation.
- 2) A six-layer verification chain that reduces citation, entailment, fidelity and consistency checks to one Verification Confidence Score, gates the answer on it, and records a per-claim proof.
- 3) An ablation study that removes one layer at a time and shows that the gate, not any single detector, is what keeps bad claims away from the user.

II. METHODOLOGY

A. Dataset and Preprocessing

We used two kinds of data. For controlled evaluation we wrote five synthetic statutes (urban tenancy, data privacy, consumer protection, small-business licensing and waste management) that follow an Act, Chapter, Section and Clause hierarchy, together with ten questions whose answers were labelled at clause level, 14 gold clauses in all. Writing the statutes ourselves meant the correct answer to every question was known exactly. To test the system on real documents we used non-disclosure agreements from ContractNLI [9] and commercial contracts from CUAD [10] and MAUD [11]. These have no statutory structure and were used for end-to-end testing, not for the scored ablations.

Text is extracted from PDF files with pypdf or read directly from plain text, then cleaned and checked for legal structure. Each upload is fingerprinted with SHA-256, and a file whose content or name matches one already in the index is rejected rather than indexed twice. Fig. 1 shows the complete flow.

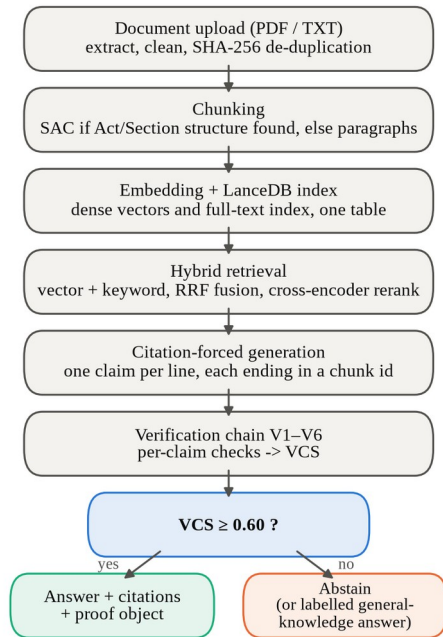


Fig. 1. Processing pipeline, from document upload to the decision to answer or abstain.

B. System Architecture

1) Adaptive chunking

When a document has a recognisable Act, Chapter, Section and Clause hierarchy, it is split along that hierarchy using summary-augmented chunking. Each chunk is embedded together with a short summary of the section it belongs to, so a clause such as “within thirty days” is not retrieved without the section that says what the thirty days apply to. The original method [4] prepends an LLM-written summary of the whole document; our summary is extractive and section-level (the section heading and its

first sentence), so ingestion needs no model call. Contracts rarely have this structure, so the system falls back to paragraph chunking and records which strategy it used.

2) Hybrid indexing and retrieval

Chunks are embedded with all-MiniLM-L6-v2 [12] and stored in LanceDB, which keeps the dense vectors and a full-text (BM25) index in a single on-disk table, so no separate database server is needed. A query runs against both indexes, and the two rankings are merged with reciprocal rank fusion [13]. A cross-encoder, bge-reranker-base [14], then rescores the merged candidates and the top k are kept. The candidate pool is deliberately much larger than k , because a reranker can only promote passages that the first stage actually returned.

3) Citation-forced generation

The prompt allows the model to use only the supplied chunks and requires one claim per line, each ending with the identifier of the chunk that supports it. If the context cannot answer the question, the model must return an abstention marker instead. This fixed format is what makes the later checks cheap, since every claim can be matched to its evidence without guessing. The generator sits behind a small adapter, so Claude, GPT, Gemini and Groq-hosted models can be swapped without changing the rest of the pipeline.

4) Verification chain

Each answer passes through six layers in order (Fig. 2). V1 confirms that every cited chunk was actually retrieved. V2 runs a natural language inference (NLI) model, roberta-large-mnli [15] trained on MultiNLI [16], with the cited passage as premise and the claim as hypothesis. V3 breaks the claim into atomic facts and checks each one, in the style of FActScore [17]. V4 is optional because it costs extra generations: it resamples the answer and measures whether the claim recurs, following SelfCheckGPT [18]. A failed V1 check or a CONTRADICTS verdict from V2 sets the claim score to zero. Otherwise the score is a weighted mean of the components that ran:

$$s = (w_e e + w_f f + w_c c) / (w_e + w_f + w_c) \quad (1)$$

where e is 1 if V2 returns ENTAILS and 0 otherwise, f is the fraction of atomic facts that are entailed, and c is the rate at which the claim recurs across resamples. The weights are $w_e = 0.40$, $w_f = 0.35$ and $w_c = 0.25$, and a layer that did not run is dropped from both the numerator and the denominator, so skipping a layer never lowers a score. V5 averages the claim scores into the VCS and applies the gate:

$$\text{VCS} = (1/N) \sum s_i; \text{ answer if } \text{VCS} \geq \tau, \text{ else abstain} \quad (2)$$

with threshold $\tau = 0.60$. V6 writes a proof object for every answer: each claim, the chunks it cites, the quoted supporting text, every layer’s verdict and the claim’s contribution to the score. The web interface shows this proof when the user clicks a citation (Fig. 3).

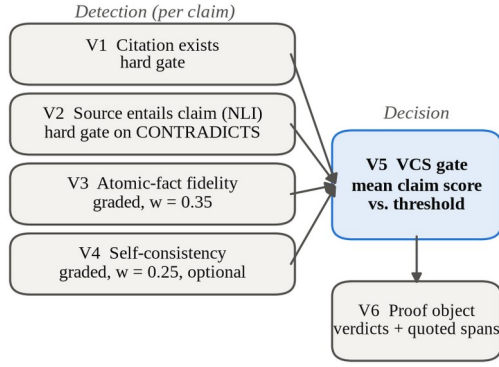


Fig. 2. The verification chain. V1–V4 score each claim; V5 turns the scores into a single decision; V6 records the evidence behind it.

5) Abstention and general-knowledge answers

Withholding an answer is the intended outcome when the documents do not support one. One case is handled differently. Questions about a general legal concept, such as “What is a non-disclosure agreement?”, are usually not defined in any indexed contract. For these the system answers from the model’s own knowledge but labels the reply as unverified and attaches no citations or score, so it cannot be mistaken for a sourced answer. Any question that refers to the indexed documents stays on the verified path.



Fig. 3. The proof view of a verified answer on a real non-disclosure agreement. The first of two claims is open, with the paragraph it cites and each layer’s verdict; V4 shows as skipped because the interface does not resample.

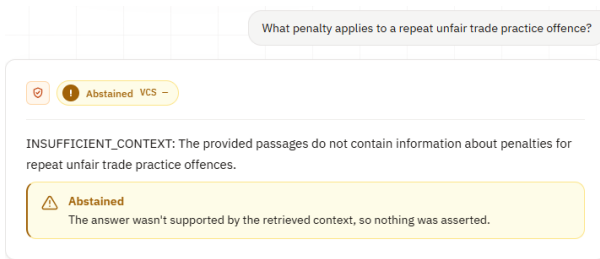


Fig. 4. An abstention. None of the indexed agreements covers repeat unfair trade practice offences, so the model returns the abstention marker and no claim is shown.

6) Web interface

The system is used through a React web interface with five pages. **Ingest** uploads a PDF or text file, reports how many chunks it produced and whether the paragraph fallback was used, and offers to replace a document that is already

indexed. **Ask** keeps saved conversations and shows every answer with its verdict, its score and clickable citations (Figs. 3 and 4). **Search** runs one query through dense, keyword and hybrid retrieval side by side, so retrieval can be inspected before any generation. **Evaluation** projects every indexed chunk to two dimensions with principal component analysis (PCA) and lists the metrics of each logged experiment (Fig. 5). **Home** shows the size of the index. The pages talk to a FastAPI backend, and every step they show is also available as an API call.

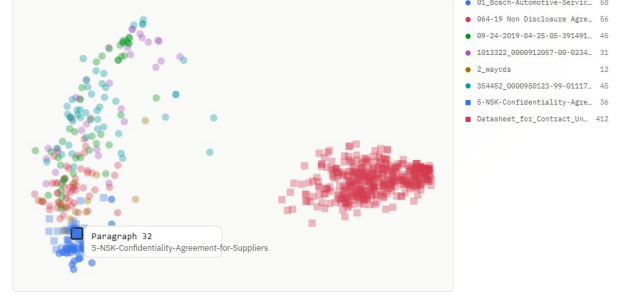


Fig. 5. The Evaluation page’s view of the index: 695 chunks from eight documents, projected with PCA and coloured by document. The 412 chunks of the one document that is not an agreement form their own cluster; the seven agreements overlap.

The overlap in Fig. 5 is expected, since the agreements share most of their vocabulary, and it is the reason every chunk identifier names its source document. A question can retrieve a similar clause from the wrong contract, and the citation makes that visible to the reader.

7) Implementation

Loading, retrieval and generation are composed with LangChain [19]; section-aware chunking and the six-layer verification chain are our own Python code. Against a plain-Python implementation of the same pipeline, the LangChain version returned identical top-5 retrieval rankings on all 10 labelled queries, with F1 0.417 and recall 1.00 for both. Replaying the same cached model answers through both, it reached identical verification decisions and scores on all 10, and neither surfaced any of 28 deliberately corrupted claims. Mean query latency differed by 4.6%, within the variation of the LLM calls.

C. Evaluation Metrics

Retrieval is scored per question against the labelled gold clauses and averaged over the question set. Precision is $P = |\text{retrieved} \cap \text{relevant}| / |\text{retrieved}|$, recall is $R = |\text{retrieved} \cap \text{relevant}| / |\text{relevant}|$, and $F1 = 2PR / (P + R)$. Retrieval rate (RR) is the share of questions for which at least one gold clause appears in the top k . SAC and paragraph chunks carry different identifiers, so relevance was matched on chunk text, which scores both chunkers by the same rule. For verification we report the mean VCS, how many questions were answered or refused, and how many known-bad claims reached the user.

D. Experimental Setup

- **Language and libraries:** Python 3.11, LangChain [19], LanceDB, sentence-transformers, Hugging Face Transformers, FastAPI and React.
- **Models:** all-MiniLM-L6-v2 (embeddings), BAAI/bge-reranker-base (reranking), roberta-large-mnli (entailment), gemini-flash-lite-latest (generation in the ablation runs and the batch test) and openai/gpt-oss-120b served through Groq (generation in the web interface).
- **Parameters:** $k = 5$, candidate pool $n = 50$, $\tau = 0.60$, weights 0.40 / 0.35 / 0.25.
- **Protocol:** each answer was generated once and every ablation arm was replayed on the cached answers, so the arms differ only in the layer removed. V4 ran on a five-question subset with two resamples each to stay within the free API quota.

III. RESULTS AND DISCUSSION

A. Retrieval and Chunking

TABLE I. Retrieval results over 10 questions ($k = 5$, $n = 50$). RR was 100% in every configuration.

Chunking	Retrieval	Rerank	P	R	F1
SAC	Dense	Yes	0.280	1.000	0.417
SAC	Dense	No	0.280	1.000	0.417
SAC	Hybrid	Yes	0.280	1.000	0.417
SAC	Hybrid	No	0.260	0.967	0.392
Paragraph	Dense	Yes	0.200	1.000	0.333
Paragraph	Dense	No	0.200	1.000	0.333
Paragraph	Hybrid	Yes	0.200	1.000	0.333
Paragraph	Hybrid	No	0.200	1.000	0.333

SAC reached a precision of 0.280 and an F1 of 0.417 in three of the four retrieval settings, against 0.200 and 0.333 for paragraph chunking in every setting (Table I, Fig. 6). Recall was at or near 1.000 throughout, so the gap comes from precision. Part of it is granularity: precision at a fixed k favours smaller chunks, and SAC passed about 550 characters of context per question to the generator against 1,500 to 1,800 for paragraph chunks. For the generator that is the useful part, since it receives a third of the text with no loss of recall.

The question set is too easy to separate the retrieval settings themselves. Every configuration found a gold clause for every question, and dense and hybrid retrieval, with or without reranking, differed by at most one relevant chunk out of fifty. We therefore draw no conclusion about them from these numbers. Reranking did have a clear cost: 6.7 to 12.8 seconds per question on our hardware, against under 0.1 seconds without it.

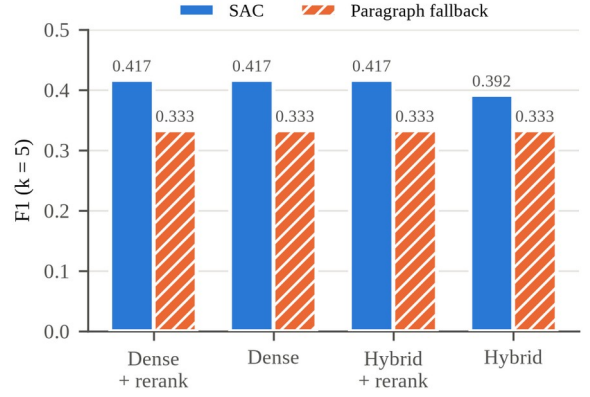


Fig. 6. Retrieval F1 for SAC and paragraph chunking under each retrieval setting.

B. Verification Ablation

To see what each layer catches, we corrupted the cached answers in two ways. In the first, every citation was replaced with a chunk identifier that was never retrieved. In the second, each claim was replaced with a claim from a different question while its citation was kept. Together these give 28 claims that must not be shown. Table II removes one layer at a time.

TABLE II. Verification ablation. Clean set: 10 questions, 14 claims. Bad claims: 28 corrupted claims across both perturbations.

Arm	Mean VCS	Answered	Refused	Bad shown
Full chain	0.768	7	3	0
– V1	0.768	7	3	0
– V2	0.813	8	2	0
– V3	0.777	7	3	0
– V4	0.723	7	3	0
– V5	0.768	10	0	28
– V6	0.768	7	3	0

The full chain showed none of the 28 corrupted claims, and neither did any arm with a single detector removed. Removing V5 showed all 28 (Fig. 7). Without the gate, the detectors still scored every corrupted claim at zero, but a score changes nothing for the user unless something acts on it. The detectors also overlap. With V1 removed, a fabricated citation still scored zero, because V2 found no premise to test and V3’s fidelity collapsed. V1’s distinct contribution is diagnostic: it is the only layer that names the missing identifiers in the proof. On clean input, V2 moved decisions the most; without it, one more question was answered and the mean VCS rose from 0.768 to 0.813. V6 changes no score by design.

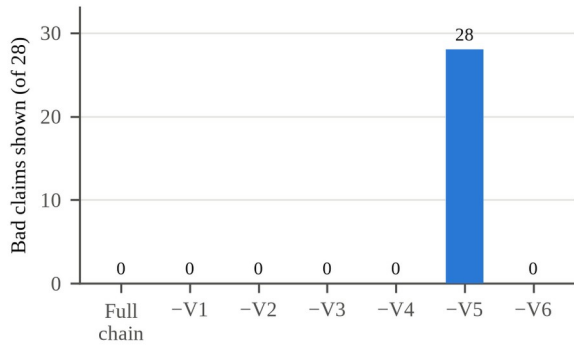


Fig. 7. Corrupted claims shown to the user when each layer is removed (28 in total). Only removing the V5 gate lets them through.

C. False Abstentions

All ten questions retrieved their gold clause, yet the full chain answered only seven. In each of the three refusals the NLI model judged a correctly cited claim NEUTRAL instead of ENTAILS; one question on grounds for eviction scored 0.0 on all three of its claims without any hard gate firing. The same pattern appeared on real contracts, described below. The 0.60 threshold is a default we have not yet calibrated, and these refusals are the clearest evidence that it needs to be.

D. Behaviour on Real Documents

We uploaded ten ContractNLI agreements through the web interface and asked five questions about specific clauses and five general questions. Rate limits on the free API tier interrupted some runs, so the results below were collected across three runs. Three of the five specific questions were verified. Citation checks were recorded for one of them, the question on the parties to the Federal Express agreement, which was verified at a VCS of 1.00 and cited that agreement; the run holding the other two ended before its citations were logged. The remaining two were refused although the answer was in the index, one of them at a VCS of 0.47. Four of the five general questions were answered from model knowledge and labelled as unverified. The fifth, “What is confidential information?”, was answered from the documents at a VCS of 0.99, because the agreements define the term. No general answer was presented as sourced.

Testing on real contracts also exposed three defects that the synthetic set never triggered, and each showed up as a silent refusal rather than an error. The candidate pool feeding the reranker was too shallow, so a correct clause ranked low in the fused list could never be promoted. A reasoning model spent most of its output budget on internal reasoning and returned an answer cut off before any complete claim. And a file name that began with a space carried the space into every chunk identifier, so V1 rejected correct citations. We found each one by inspecting the intermediate output (the retrieved list, the token counts, the identifiers) before changing any code.

Fig. 3 exposes a weakness of averaging. The answer’s VCS of 0.62 is the mean of two claim scores, and the first claim

contributes 1.00, so the second scored below 0.25. It was shown only because a strong claim sat next to it. Gating each claim on its own score would have withheld it.

IV. AUTHORS AND METRICS

A. Authors and Affiliations

[Authors, department and guide.]

B. Figures and Tables

Figs. 6 and 7 were plotted with Matplotlib directly from the result files logged for each experiment, and every number in Tables I and II comes from those files. Figs. 1 and 2 were drawn for this paper, and Figs. 3–5 are screenshots of the web interface taken during real use.

V. CHALLENGES AND LIMITATIONS

- **Small evaluation set:** ten questions, 14 gold clauses and five synthetic statutes, with one model of each kind.
- **Saturated retrieval benchmark:** every configuration found a gold clause for every question, so the set cannot rank retrieval methods.
- **Uncalibrated threshold:** $\tau = 0.60$ is a default, and three of ten answerable questions were refused.
- **Synthetic corruptions:** the corrupted claims show what the chain catches, not how often the model hallucinates unprompted; that rate was not measured.
- **General-domain NLI:** roberta-large-mnli was not trained on legal text, and its NEUTRAL verdicts caused the false refusals.
- **Mean aggregation:** averaging lets a weak claim through when it sits next to a strong one, as in Fig. 3.
- **Latency and quotas:** reranking added several seconds per question, and free-tier rate limits interrupted batch runs.

VI. FUTURE ENHANCEMENTS

- Gate each claim on its own score, or aggregate with the minimum instead of the mean, so one strong claim cannot carry a weak one.
- Calibrate the threshold on a held-out set that includes unanswerable questions, so correct refusals can be counted as well as false ones.
- Evaluate on LegalBench-RAG and on Indian statutes from India Code, the corpus this project originally set out to serve.
- Fine-tune or replace the NLI model with one trained on legal text to reduce NEUTRAL verdicts on correct citations.
- Build a question set with answers split across adjacent chunks to measure neighbour-chunk expansion, which is implemented but off by default.

- Add HTML ingestion so the full ContractNLI release can be indexed.

ACKNOWLEDGMENT

[Acknowledgment.]

REFERENCES

- [1] *Mata v. Avianca, Inc.*, No. 22-cv-1461 (PKC) (S.D.N.Y. 2023).
- [2] Magesh, V., et al. (2025). *Hallucination-Free? Assessing the Reliability of Leading AI Legal Research Tools*. Journal of Empirical Legal Studies.
- [3] Lewis, P., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. Advances in Neural Information Processing Systems (NeurIPS).
- [4] Reuter, M., Lingenberg, T., Liepina, R., Lagioia, F., Lippi, M., Sartor, G., Passerini, A., & Sayin, B. (2025). *Towards Reliable Retrieval in RAG Systems for Large Legal Datasets*. Proceedings of the Natural Legal Language Processing Workshop (NLLP), pp. 17–30.
- [5] Figueiredo, A., Lucas, J., Ribeiro, T., et al. (2026). *Grounded in Law: A Multi-Stage Anti-Hallucination Pipeline for Legal RAG Systems in Brazilian Portuguese*. Proceedings of the 17th International Conference on Computational Processing of Portuguese (PROPOR).
- [6] Maghakian, J., Sinha, R., Schettewi, M., & Kaur, G. (2025). *Embedding-Free RAG*. Findings of the Association for Computational Linguistics: EMNLP 2025, pp. 24974–24985.
- [7] Niu, C., et al. (2024). *RAGTruth: A Hallucination Corpus for Developing Trustworthy Retrieval-Augmented Language Models*. Annual Meeting of the Association for Computational Linguistics (ACL).
- [8] Pipitone, N., & Alami, G. H. (2024). *LegalBench-RAG: A Benchmark for Retrieval-Augmented Generation in the Legal Domain*. arXiv.
- [9] Koreeda, Y., & Manning, C. D. (2021). *ContractNLI: A Dataset for Document-level Natural Language Inference for Contracts*. Findings of EMNLP.
- [10] Hendrycks, D., Burns, C., Chen, A., & Ball, S. (2021). *CUAD: An Expert-Annotated NLP Dataset for Legal Contract Review*. NeurIPS Datasets and Benchmarks Track.
- [11] Wang, S. H., et al. (2023). *MAUD: An Expert-Annotated Legal NLP Dataset for Merger Agreement Understanding*. Conference on Empirical Methods in Natural Language Processing (EMNLP).
- [12] Reimers, N., & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. EMNLP-IJCNLP.
- [13] Cormack, G. V., Clarke, C. L. A., & Büttcher, S. (2009). *Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods*. ACM SIGIR Conference.
- [14] Xiao, S., et al. (2023). *C-Pack: Packaged Resources to Advance General Chinese Embedding*. arXiv:2309.07597.
- [15] Liu, Y., et al. (2019). *RoBERTa: A Robustly Optimized BERT Pretraining Approach*. arXiv:1907.11692.
- [16] Williams, A., Nangia, N., & Bowman, S. R. (2018). *A Broad-Coverage Challenge Corpus for Sentence Understanding through Inference*. NAACL-HLT.
- [17] Min, S., et al. (2023). *FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation*. Conference on Empirical Methods in Natural Language Processing (EMNLP).
- [18] Manakul, P., Liusie, A., & Gales, M. J. F. (2023). *SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models*. Conference on Empirical Methods in Natural Language Processing (EMNLP).
- [19] Chase, H. (2022). *LangChain* [Computer software]. <https://github.com/langchain-ai/langchain>.